

Laboratório de Concorrência · Aula 10

Luiz Augusto Cavalcante de Holanda · RA 24027661-2 · Turma ESOF6S-N-D

Gerado em 09/10/2026, 21:26:29 · tempo registrado no cronômetro: 71:53

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Na sincronização de cooperação uma tarefa precisa esperar que a outra termine algo, ou seja, há dependência de ordem. Exemplo: Corrida de revezamento. Já sincronização de competição, as tarefas competem por um mesmo recurso, que só pode ser usado por uma de cada vez. Exemplo: Dois carros chegando juntos em uma ponte de pista única.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2.000.000.

1ª execução (n = 1.000.000)

1358020

2ª execução (n = 1.000.000)

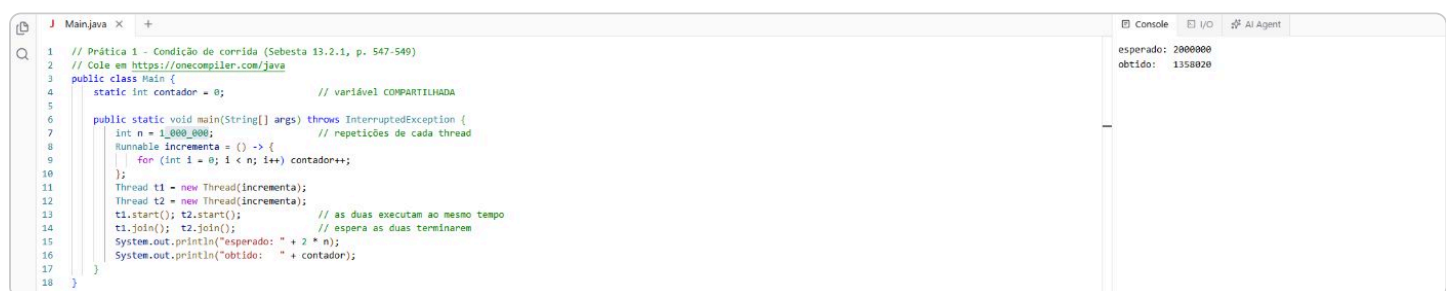
978593

3ª execução (n = 1.000.000)

não respondido

Saída do programa

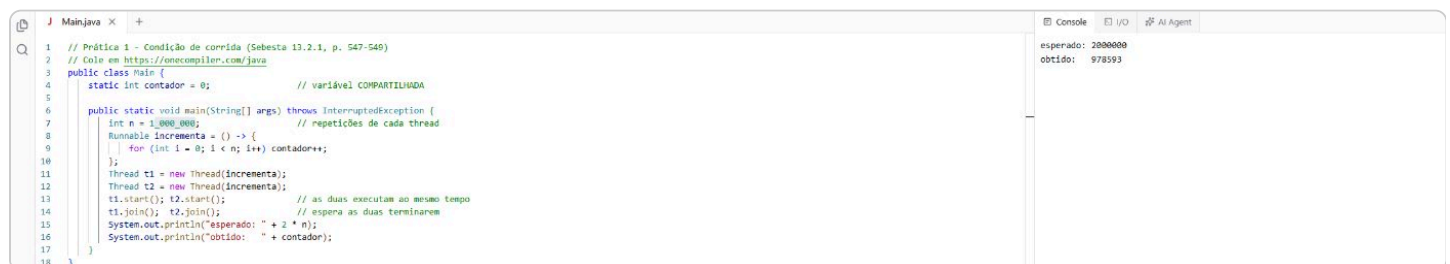
não respondido



```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

Console I/O AI Agent

esperado: 2000000
obtido: 1358020



```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

Console I/O AI Agent

esperado: 2000000
obtido: 978593

```

1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }

```

Console: esperado: 2000000
obtido: 1014002

```

1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }

```

Console: esperado: 2000
obtido: 2000

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Os valores não bateram com o esperado e não foram repetidos. Isso ocorre pois o escalonador decide a intercalação das threads a cada execução e o número de atualizações perdidas depende dessa intercalação.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Se contador = 10:

1. A lê 10.
2. B lê 10, antes de A gravar.
3. A soma e grava 11.
4. Soma (10 + 1) e grava 11.

Foram feitas duas somas, mas o contador só subiu 1. Assim a atualização de A foi sobrescrita e se perde.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

O resultado bateu com o esperado nas duas vezes, devido ao tempo de trabalho ser curto e fazer com que uma thread termine antes de outra começar. Esse erro é perigoso porque pode não ser detectado durante os testes, mas aparece em um ambiente de produção com muitos usuários e carga alta, sendo difícil de reproduzir e depurar.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

- AAABBB → 8
- BABAAB → 6
- ABBABA → 4
- BBAABA → 4
- BABBAA → 4
- BAABBA → 4
- ABAABB → 6
- BBBAAA → 7

Saída do programa

não respondido

```

1 # Prática 2 - Todas as respostas possíveis da Figura 13.1 (Sobesta, p. 548)
2 # Cole em https://onecompiler.com/python
3 # TOTAL começa em 2. A tarefa A faz TOTAL + 1 e a tarefa B faz TOTAL * 2.
4 # Cada tarefa tem 3 passos: 1) ler TOTAL; 2) calcular; 3) gravar TOTAL.
5 from itertools import combinations
6
7
8 def executa(ordem):
9     # ordem: por exemplo "AABBBB"
10    total = 2
11    reg = ["A": 0, "B": 0]
12    # o "registrador" de cada tarefa
13    passo = ["A": 0, "B": 0]
14    for t in ordem:
15        if passo[t] == 0:
16            reg[t] = total
17        elif passo[t] == 1:
18            reg[t] = reg[t] + 1 if t == "A" else reg[t] * 2
19        else:
20            total = reg[t]
21        passo[t] += 1
22    return total
23
24 print("AABBBB ->", executa("AABBBB")) # A lê 2, depois B lê 2
25 print("BBBBAA ->", executa("BBBBAA")) # B lê 2, depois A lê 2
26 print("AABBAAB ->", executa("AABBAAB")) # alternando passo a passo
27
28 resultados = {}
29 for posicoes in combinations(range(0, 8)):
30     # onde ficam os 3 passos de A
31     ordens = "".join("A" if i in posicoes else "B" for i in range(8))
32     resultados.setdefault(executa(ordens), []).append(ordens)
33
34 print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
35 for valor in sorted(resultados):
36     ordens = resultados[valor]
37     print(f"TOTAL = {valor}: {len(ordens)} ordens, por exemplo {ordens[0]}")
  
```

Console

```

AABBBB -> 8
BBBBAA -> 7
AABBAAB -> 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 6 ordens, por exemplo AABBAAB
TOTAL = 7: 1 ordem, por exemplo BBBBAA
TOTAL = 8: 1 ordem, por exemplo AABBBB
  
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São possíveis 20 ordens com 4 resultados diferentes. Só uma ordem dá 8, AABBBB.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Ordem: AABBBB

1. A lê 3.
2. A calcula $3 + 1 = 4$ no registrador.
3. B lê 3.
4. B calcula 6.
5. B grava TOTAL = 6.
6. A grava TOTAL = 4.

A operação de B foi sobrescrito pela gravação de A.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Eu acredito que a mais rápida é a com operação atômica. Talvez a com o semáforo pode causar uma saída errada não esperada.

Saída do programa

não respondido

```

1 // Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sobesta 13.4 e 13.7, p. 557-578)
2 // Cole em https://onecompiler.com/java
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class Main {
6     static int contadores = 0, contSync = 0;
7     static AtomicInteger atomica = new AtomicInteger();
8
9     static synchronized void incSync() { contSync++; } // uma thread por vez
10
11     static long runda(Runnable corpo) throws InterruptedException {
12         Thread a = new Thread(corpo); b = new Thread(corpo);
13         long inicio = System.nanoTime();
14         a.start(); b.start(); a.join(); b.join();
15         return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
16     }
17
18     public static void main(String[] args) throws InterruptedException {
19         int n = 1_000_000;
20         long m1 = modo1() -> { for (int i = 0; i < n; i++) contCuidados++; };
21         long m2 = modo2() -> { for (int i = 0; i < n; i++) incSync(); };
22         long m3 = modo3() -> { for (int i = 0; i < n; i++) atomica.incrementAndGet(); };
23         System.out.println("esperado: " + 2 * n);
24         System.out.println("sem sincronizacao: " + contCuidados + " em " + m1 + " ms");
25         System.out.println("sincronizado: " + contSync + " em " + m2 + " ms");
26         System.out.println("atomicInteger: " + atomica.get() + " em " + m3 + " ms");
27     }
28 }
  
```

Console

```

esperado: 2000000
sem sincronizacao: 9820732 em 2 ms
sincronizado: 2000000 em 85 ms
atomicInteger: 2000000 em 71 ms
  
```

```

1 // Prática 3 - Corrigindo a corrida de dados e operação atômica (Sebesta 13.4 e 13.7, p. 557-578)
2 // Cole em https://onecompiler.com/java
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class Main {
6     static int seqCuidado = 0, contSync = 0;
7     static AtomicInteger atomico = new AtomicInteger();
8
9     static synchronized void incSync() { contSync++; } // uma thread por vez
10
11     static long node(Runnable corpo) throws InterruptedException {
12         Thread a = new Thread(corpo); b = new Thread(corpo);
13         long inicio = System.nanoTime();
14         a.start(); b.start(); a.join(); b.join();
15         return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
16     }
17
18     public static void main(String[] args) throws InterruptedException {
19         int n = 1_000_000;
20         long ms1 = node(() -> { for (int i = 0; i < n; i++) seqCuidado++; });
21         long ms2 = node(() -> { for (int i = 0; i < n; i++) incSync(); });
22         long ms3 = node(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
23         System.out.println("esperado: " + 2 * n);
24         System.out.println("sem sincronizacao: " + seqCuidado + " em " + ms1 + " ms");
25         System.out.println("synchronized: " + contSync + " em " + ms2 + " ms");
26         System.out.println("AtomicInteger: " + atomico.get() + " em " + ms3 + " ms");
27     }
28 }

```

Console:

```

esperado: 2000000
sem sincronizacao: 10/4100 em 5 ms
synchronized: 2000000 em 80 ms
AtomicInteger: 2000000 em 15 ms

```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

Deram certo as versões com synchronized e AtomicInteger. A versão correta é mais lenta as threads precisam se revezar, pois há espera pelo bloqueio, disputa pelo recurso e trocas de contexto. A versão sem sincronização é mais rápida justamente porque erra.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não resolve, pois AtomicInteger só garante atomicidade de uma operação isolada. Uma transferência bancária exige o debitar e creditar e nesse processo pode haver falha ou um estado inconsistente. Para resolver seria preciso tornar a transferência inteira uma seção crítica, ou seja, usar o synchronized para cobrir as duas contas ou ainda usar uma transaction de banco de dados.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

As mensagens "A terminou" e "B terminou" não vão aparecer. A pega X e espera Y, enquanto B pega Y e espera X e as duas ficam presas.

Saídas (antes e depois da correção)

não respondido

```

1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) {}
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
15        });
16        Thread b = new Thread(() -> {
17            synchronized (Y) { pausa(); synchronized (X) { System.out.println("B terminou"); } }
18        });
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse" : "IMPASSE: " + presas.length + " threads presas");
23        System.exit(0); // encerra o programa
24    }
25 }

```

Console:

```

IMPASSE: 2 threads presas

```

```

1 // Prática 4 - Impasse (Deadlock) (Sebesta 13.2.1, p. 551)
2 // cole em https://mccompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(); Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
15        };
16        Thread b = new Thread() -> {
17            synchronized (X) { pausa(); synchronized (Y) { System.out.println("B terminou"); } }
18        };
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse" : "IMPASSE: " + presas.length + " threads presas");
23
24        // encerra o programa
25        System.exit(0);
26    }
27 }

```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A tinha o X e esperava o Y. B tinha o Y e esperava o X. Cada uma segura o que a outra precisa e nenhuma libera, então esperam para sempre (espera circular).

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Não existe mais ciclo de espera, pois com a mesma ordem quem pega o X primeiro fica com os dois recursos e a outra espera o X sem segurar nada. Exemplo: banco de dados com duas transações que atualizam as mesmas duas linhas em ordens opostas.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

3 trabalhadores: cerca de 200ms.

1 trabalhador: cerca de 600ms.

6 trabalhadores: cerca de 100ms.

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

não respondido

```

1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 3 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }

```

Console

```

trabalhador 3 fez a tarefa 1
trabalhador 2 fez a tarefa 3
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 6
trabalhador 2 fez a tarefa 5
trabalhador 3 fez a tarefa 4
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

```

```

1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 3 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }

```

Console

```

trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms

```

```

1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 6 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }

```

Console

```

trabalhador 2 fez a tarefa 3
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
trabalhador 5 fez a tarefa 6
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms

```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Os tempos seguem de dividir 6 tarefas de 100 ms entre os trabalhadores: 6×100 , 2×100 e 1×100 . O programa não precisa de trava pois não há dado compartilhado e modificável.

Saídas (antes e depois da troca)

não respondido

```

1 // Prática 5b - Um canal sem bloqueio do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int) // canal 10M espaço (sem buffer)
9     ch <- 1 // envia... e espera alguém receber
10     fmt.Println(<ch>)
11 }

```

Console

```

fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
  main.main()
    /home/luiz2/.go/src/main.go:10:13
exit status 2

```

```
1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int, 1) // canal SEM espaço (sem buffer)
9     ch <- 1                 // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Trava porque em um canal sem buffer, quem envia fica esperando um receptor, mas a única goroutine está presa no envio e nunca chaga ao <-ch. Com buffer de 1, a mensagem fica guardada no canal, o envio termina sem esperar e depois o main recebe.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Não compila, pois o compilador reclama que a closure pode viver mais que a função e que pega emprestada a variável contador, e sugere usar move.

Saídas (o erro e o resultado com move)

não respondido

```
1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(|| {
8         contador += 1; // a thread altera a variável de main
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }
```

```
error[E0373]: closure may outlive the current function, but it borrows 'contador', which is owned by the current function
--> HelloWorld.rs:7:27
7 |     let t = thread::spawn(|| {
8 |         contador += 1; // a thread altera a variável de main
9 |     });
    |         ^^^^^^^^^ 'contador' is borrowed here
note: function requires argument type to outlive "'static'
--> HelloWorld.rs:7:13
7 |     let t = thread::spawn(|| {
8 |         contador += 1; // a thread altera a variável de main
9 |     });
    |         ^
help: to force the closure to take ownership of 'contador' (and any other referenced variables), use the 'move' keyword
7 |     let t = thread::spawn(move || {
8 |         contador += 1; // a thread altera a variável de main
9 |     });
    |         ^^^^^
error[E0502]: cannot borrow 'contador' as mutable because it is also borrowed as immutable
--> HelloWorld.rs:11:20
11 |     println!("{}", contador);
    |                    ^^^^^^^^^ immutable borrow occurs here
7 |     let t = thread::spawn(|| {
8 |         contador += 1; // a thread altera a variável de main
9 |     });
    |         ^^^^^^^^^ first borrow occurs due to use of 'contador' in closure
10 |    t.join().unwrap();
11 |    println!("{}", contador);
    |                    ^^^^^^^^^ mutable borrow occurs here
note: requirement that the value outlives "'static' introduced here
--> /rustc/48e222cf44895c899001411808050a9905/library/std/src/thread/function.rs:128:14
error: aborting due to 2 previous errors

Some errors have detailed explanations: E0373, E0502.
For more information about an error, try 'rustc --explain E0373'.
Debug this error
```

```
1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(move || {
8         contador += 1; // a thread altera a cópia da variável
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }
```

```
warning: value captured by 'contador' is never read
--> HelloWorld.rs:8:9
8 |         contador += 1; // a thread altera a cópia da variável
  |         ^^^^^^^^^^^^^
help: did you mean to capture by reference instead?
= note: #[warn(unused_assignments)] (part of #[warn(unused)]) on by default

warning: value assigned to 'contador' is never read
--> HelloWorld.rs:8:9
8 |         contador += 1; // a thread altera a cópia da variável
  |         ^^^^^^^^^^^^^
help: maybe it is overwritten before being read?

warning: 2 warnings emitted
0
```

Saída do programa

não respondido

```

1 // Prática 0 - Rust: compartilhar com Arc (contagem de referências) e Mutex (exclusão mútua)
2 // Cole em https://onecompiler.com/rust
3 use std::sync::{Arc, Mutex};
4 use std::thread;
5
6 fn main() {
7     let contador = Arc::new(Mutex::new(0)); // compartilhado e protegido
8     let mut threads = Vec::new();
9     for _ in 0..4 {
10         let c = Arc::clone(&contador);
11         threads.push(thread::spawn(move || {
12             for _ in 0..100_000 {
13                 *c.lock().unwrap() += 1; // trava, soma e destrava
14             }
15         }));
16     }
17     for t in threads { t.join().unwrap(); }
18     println!("contador = {}", *contador.lock().unwrap());
19 }

```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move, a thread leva uma cópia, incrementa a cópia dela e a variável de main nunca muda, então imprime 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java o erro só aparece em execução, de forma intermitente, às vezes só em produção. Já em Rust o erro é descoberto na compilação. A vantagem é que a segurança é garantida antes de o programa rodar. O custo é uma curva de aprendizado maior, brigar com o compilador (ownership e borrow), código mais verboso e rigidez.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A ordem será A, D, C, B

Saída do programa

não respondido

```

1 // Prática 7 - JavaScript: uma única thread e um laço de eventos
2 // Cole em https://onecompiler.com/nodejs
3 console.log("A");
4 setTimeout(() => console.log("B"), 0);
5 Promise.resolve().then(() => console.log("C"));
6 console.log("D");
7
8 function ocupado(ms) { // laço que NÃO devolve o controle
9     const fim = Date.now() + ms;
10    while (Date.now() < fim) { }
11 }
12
13 setTimeout(() => {
14     const t0 = Date.now();
15     setTimeout(() => console.log("E: timer de 10 ms rodou depois de " + (Date.now() - t0) + " ms"), 10);
16     ocupado(300); // trava a única thread por 300 ms
17 }, 0);

```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

Ordem: A, D, C, B.

O código síncrono executa primeiro A e D, depois vêm as promessas C, na fila de microtarefas. Por fim vem os

timers B, mesmo com tempo 0, na fila de tarefas, que só roda quando a pilha esvazia e as microtarefas acabam.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Porque havia só uma thread, e ela estava ocupada pelo laço ocupado(300). Em um servidor Node, um cálculo pesado bloqueia todos os usuários, ou seja, nenhuma outra requisição é atendida.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode ocorrer uma condição de corrida: as duas leem “1 lugar disponível” e as duas confirmam a compra, ou seja, venda duplicada. Para evitar torne “verificar e reservar” uma operação atômica, por exemplo com uma transação no banco.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O erro quase sumir com $n = 1000$.